VHDL cheatsheet

VHDL (VHSIC Hardware Description Language) cheat sheet - A comprehensive reference for digital hardware design and simulation.

Star on GitHub 9.6k
Follow us on X
Buy Me a Coffee

# Getting Started

Introduction

VHDL is a hardware description language used for electronic design automation to describe digital and mixed-signal systems.

- IEEE 1076 Standard (IEEE Standard)
- VHDL Tutorial (nandland.com)
- Digital Design VHDL (lapinoo.net)

Basic Entity

```
entity hello_world is
end hello_world;

architecture behavioral of hello_world is
begin
  -- Simple testbench process
  process
  begin
    report "Hello, VHDL World!";
    wait;
  end process;
end behavioral;
```

A basic VHDL entity and architecture

Simple AND Gate

```
library ieee;
use ieee.std_logic_1164.all;

entity and_gate is
  port (
    a : in  std_logic;
    b : in  std_logic;
    y : out std_logic
  );
end and_gate;

architecture behavioral of and_gate is
begin
  y <= a and b;
end behavioral;
```

Basic two-input AND gate implementation

Libraries and Packages

```
library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;
use ieee.std_logic_arith.all;
use ieee.std_logic_unsigned.all;

library work;
use work.my_package.all;
```

Common library declarations

Comments

```
-- Single line comment

/* Multi-line comment
   spanning multiple lines
   (VHDL-2008 feature) */
```

# Data Types

Standard Logic Types

`bit`	Binary bit	'0', '1'
`std_logic`	IEEE standard	'0', '1', 'Z', 'X', 'H', 'L', 'W', 'U', '-'
`boolean`	Boolean type	true, false
`integer`	Integer type	-2^31 to 2^31-1
`real`	Real number	Floating point
`character`	Single character	ASCII characters
`string`	Character array	"Hello World"

Vector Types

```
-- Bit vectors
signal data_bus : bit_vector(7 downto 0);
signal address  : bit_vector(15 downto 0);

-- Standard logic vectors
signal data     : std_logic_vector(7 downto 0);
signal control  : std_logic_vector(3 downto 0);

-- Signed and unsigned (numeric_std)
signal counter  : unsigned(7 downto 0);
signal offset   : signed(15 downto 0);
```

Array Types

```
-- Array type declaration
type memory_type is array (0 to 255) of std_logic_vector(7 downto 0);

-- Array signal declaration
signal memory : memory_type;

-- Multi-dimensional arrays
type matrix_type is array (0 to 3, 0 to 3) of integer;
signal matrix : matrix_type;
```

Record Types

```
-- Record type declaration
type cpu_interface is record
  address : std_logic_vector(15 downto 0);
  data    : std_logic_vector(7 downto 0);
  read_en : std_logic;
  write_en: std_logic;
end record;

-- Record signal
signal cpu_bus : cpu_interface;

-- Access record fields
cpu_bus.address <= x"1234";
cpu_bus.data    <= x"AB";
```

# Entity and Architecture

Entity Declaration

```
entity counter is
  generic (
    WIDTH : integer := 8;
    MAX_COUNT : integer := 255
  );
  port (
    clk     : in  std_logic;
    reset   : in  std_logic;
    enable  : in  std_logic;
    count   : out std_logic_vector(WIDTH-1 downto 0);
    overflow: out std_logic
  );
end entity counter;
```

Entity with generics and ports

Architecture Styles

```
-- Behavioral Architecture
architecture behavioral of counter is
  signal count_int : unsigned(WIDTH-1 downto 0);
begin
  process(clk, reset)
  begin
    if reset = '1' then
      count_int <= (others => '0');
      overflow <= '0';
    elsif rising_edge(clk) then
      if enable = '1' then
        if count_int = MAX_COUNT then
          count_int <= (others => '0');
          overflow <= '1';
        else
          count_int <= count_int + 1;
          overflow <= '0';
        end if;
      end if;
    end if;
  end process;

  count <= std_logic_vector(count_int);
end behavioral;
```

Structural Architecture

```
architecture structural of adder_4bit is
  component full_adder is
    port (
      a, b, cin : in  std_logic;
      sum, cout : out std_logic
    );
  end component;

  signal carry : std_logic_vector(3 downto 0);
begin
  -- Component instantiation
  FA0: full_adder port map (a(0), b(0), cin, sum(0), carry(0));
  FA1: full_adder port map (a(1), b(1), carry(0), sum(1), carry(1));
  FA2: full_adder port map (a(2), b(2), carry(1), sum(2), carry(2));
  FA3: full_adder port map (a(3), b(3), carry(2), sum(3), cout);
end structural;
```

Dataflow Architecture

```
architecture dataflow of mux_4to1 is
begin
  -- Concurrent signal assignments
  output <= input0 when sel = "00" else
            input1 when sel = "01" else
            input2 when sel = "10" else
            input3;

  -- Conditional assignment
  with sel select
    output <= input0 when "00",
              input1 when "01",
              input2 when "10",
              input3 when others;
end dataflow;
```

# Processes and Sequential Logic

Clocked Process

```
-- Synchronous process with clock and reset
process(clk, reset)
begin
  if reset = '1' then
    -- Asynchronous reset
    output <= '0';
    state <= IDLE;
  elsif rising_edge(clk) then
    -- Synchronous logic
    case state is
      when IDLE =>
        if start = '1' then
          state <= ACTIVE;
        end if;
      when ACTIVE =>
        output <= '1';
        if done = '1' then
          state <= IDLE;
        end if;
    end case;
  end if;
end process;
```

Combinational Process

```
-- Combinational logic process
process(a, b, sel)
begin
  case sel is
    when "00" =>
      result <= a and b;
    when "01" =>
      result <= a or b;
    when "10" =>
      result <= a xor b;
    when others =>
      result <= not a;
  end case;
end process;
```

Wait Statements

```
-- Testbench process with wait statements
process
begin
  -- Initialize
  reset <= '1';
  wait for 10 ns;

  reset <= '0';
  wait for 20 ns;

  -- Apply stimulus
  data_in <= x"AA";
  wait until rising_edge(clk);

  -- Wait for condition
  wait until ready = '1';

  wait;  -- Wait indefinitely
end process;
```

# Operators and Expressions

Logical Operators

```
-- Logical operators
result <= a and b;
result <= a or b;
result <= a nand b;
result <= a nor b;
result <= a xor b;
result <= a xnor b;
result <= not a;
```

Arithmetic Operators

```
-- Arithmetic operators (requires numeric_std)
signal a, b : unsigned(7 downto 0);
signal result : unsigned(8 downto 0);

result <= ('0' & a) + ('0' & b);  -- Addition with carry
result <= a - b;                   -- Subtraction
result <= a * b;                   -- Multiplication
result <= a / b;                   -- Division (synthesis limited)
result <= a mod b;                 -- Modulo
result <= a rem b;                 -- Remainder
result <= a ** 2;                  -- Exponentiation

-- Increment/Decrement
counter <= counter + 1;
counter <= counter - 1;
```

Relational Operators

```
-- Comparison operators
if a = b then         -- Equal
if a /= b then        -- Not equal
if a < b then         -- Less than
if a <= b then        -- Less than or equal
if a > b then         -- Greater than
if a >= b then        -- Greater than or equal
```

Shift Operators

```
-- Shift operators (VHDL-2008)
signal data : std_logic_vector(7 downto 0);

data <= data sll 1;   -- Shift left logical
data <= data srl 1;   -- Shift right logical
data <= data sla 1;   -- Shift left arithmetic
data <= data sra 1;   -- Shift right arithmetic
data <= data rol 1;   -- Rotate left
data <= data ror 1;   -- Rotate right
```

# Control Structures

If-Then-Else

```
-- Simple if statement
if enable = '1' then
  output <= input;
end if;

-- If-else
if reset = '1' then
  counter <= 0;
else
  counter <= counter + 1;
end if;

-- If-elsif-else
if sel = "00" then
  output <= input0;
elsif sel = "01" then
  output <= input1;
elsif sel = "10" then
  output <= input2;
else
  output <= input3;
end if;
```

Case Statement

```
-- Case statement
case state is
  when IDLE =>
    next_state <= START;
    output <= '0';
  when START =>
    next_state <= ACTIVE;
    output <= '1';
  when ACTIVE =>
    if done = '1' then
      next_state <= IDLE;
    else
      next_state <= ACTIVE;
    end if;
  when others =>
    next_state <= IDLE;
    output <= '0';
end case;
```

For Loops

```
-- For loop in process
process(clk)
begin
  if rising_edge(clk) then
    for i in 0 to 7 loop
      shift_reg(i+1) <= shift_reg(i);
    end loop;
    shift_reg(0) <= serial_in;
  end if;
end process;

-- Generate statement with for loop
gen_mux: for i in 0 to 3 generate
  mux_inst: entity work.mux_2to1
    port map (
      a => input_a(i),
      b => input_b(i),
      sel => sel,
      y => output(i)
    );
end generate;
```

While Loops

```
-- While loop (primarily for testbenches)
process
  variable count : integer := 0;
begin
  while count < 10 loop
    wait until rising_edge(clk);
    count := count + 1;
  end loop;
  wait;
end process;
```

# Functions and Procedures

Function Declaration

```
-- Function declaration
function to_integer(input : std_logic_vector) return integer is
  variable result : integer := 0;
begin
  for i in input'range loop
    result := result * 2;
    if input(i) = '1' then
      result := result + 1;
    end if;
  end loop;
  return result;
end function;

-- Function usage
signal address : std_logic_vector(7 downto 0);
signal addr_int : integer;

addr_int <= to_integer(address);
```

Procedure Declaration

```
-- Procedure declaration
procedure write_memory(
  signal clk : in std_logic;
  signal addr : in std_logic_vector(7 downto 0);
  signal data : in std_logic_vector(7 downto 0);
  signal mem : inout memory_type
) is
begin
  wait until rising_edge(clk);
  mem(to_integer(unsigned(addr))) <= data;
end procedure;

-- Procedure call
write_memory(clk, address, data_in, memory);
```

Package Declaration

```
-- Package declaration
package my_package is
  -- Constants
  constant DATA_WIDTH : integer := 8;
  constant ADDR_WIDTH : integer := 16;

  -- Types
  type state_type is (IDLE, READ, WRITE, DONE);

  -- Functions
  function parity(data : std_logic_vector) return std_logic;

  -- Procedures
  procedure reset_signals(signal clk : in std_logic);
end package;

-- Package body
package body my_package is
  function parity(data : std_logic_vector) return std_logic is
    variable result : std_logic := '0';
  begin
    for i in data'range loop
      result := result xor data(i);
    end loop;
    return result;
  end function;
end package body;
```

# Testbenches and Simulation

Basic Testbench

```
library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;

entity tb_counter is
end tb_counter;

architecture behavioral of tb_counter is
  -- Component declaration
  component counter is
    port (
      clk     : in  std_logic;
      reset   : in  std_logic;
      enable  : in  std_logic;
      count   : out std_logic_vector(7 downto 0)
    );
  end component;

  -- Testbench signals
  signal clk     : std_logic := '0';
  signal reset   : std_logic := '1';
  signal enable  : std_logic := '0';
  signal count   : std_logic_vector(7 downto 0);

  -- Clock period
  constant clk_period : time := 10 ns;

begin
  -- Unit Under Test instantiation
  uut: counter
    port map (
      clk    => clk,
      reset  => reset,
      enable => enable,
      count  => count
    );

  -- Clock generation
  clk_proc: process
  begin
    clk <= '0';
    wait for clk_period/2;
    clk <= '1';
    wait for clk_period/2;
  end process;

  -- Stimulus process
  stim_proc: process
  begin
    -- Reset phase
    reset <= '1';
    wait for 20 ns;
    reset <= '0';

    -- Enable counting
    enable <= '1';
    wait for 100 ns;

    -- Disable counting
    enable <= '0';
    wait for 50 ns;

    -- End simulation
    wait;
  end process;
end behavioral;
```

Assert Statements

```
-- Assert for verification
process(clk)
begin
  if rising_edge(clk) then
    -- Check counter overflow
    assert count /= "11111111"
      report "Counter overflow detected!"
      severity warning;

    -- Check reset functionality
    if reset = '1' then
      assert count = "00000000"
        report "Reset failed!"
        severity error;
    end if;
  end if;
end process;
```

File I/O

```
-- File I/O for testbenches
use std.textio.all;

process
  file input_file  : text open read_mode is "input.txt";
  file output_file : text open write_mode is "output.txt";
  variable line_in  : line;
  variable line_out : line;
  variable data     : integer;
begin
  while not endfile(input_file) loop
    readline(input_file, line_in);
    read(line_in, data);

    -- Process data
    data := data * 2;

    write(line_out, data);
    writeline(output_file, line_out);
  end loop;

  wait;
end process;
```

# Attributes and Configuration

Signal Attributes

```
-- Common signal attributes
signal data : std_logic_vector(7 downto 0);

-- Range attributes
data'left      -- 7 (leftmost index)
data'right     -- 0 (rightmost index)
data'high      -- 7 (highest index)
data'low       -- 0 (lowest index)
data'range     -- 7 downto 0
data'length    -- 8 (number of elements)

-- Event attributes
clk'event      -- true if clk changed
clk'last_event -- time since last event
data'stable(5 ns) -- true if stable for 5 ns
```

Type Attributes

```
-- Type attributes
type state_type is (IDLE, ACTIVE, DONE);

state_type'left        -- IDLE
state_type'right       -- DONE
state_type'pos(ACTIVE) -- 1
state_type'val(2)      -- DONE
state_type'succ(IDLE)  -- ACTIVE
state_type'pred(DONE)  -- ACTIVE
```

Generate Statements

```
-- Conditional generate
gen_async: if ASYNC_RESET generate
  process(clk, reset)
  begin
    if reset = '1' then
      output <= '0';
    elsif rising_edge(clk) then
      output <= input;
    end if;
  end process;
end generate;

gen_sync: if not ASYNC_RESET generate
  process(clk)
  begin
    if rising_edge(clk) then
      if reset = '1' then
        output <= '0';
      else
        output <= input;
      end if;
    end if;
  end process;
end generate;
```

# Advanced Concepts

State Machines

```
-- Finite State Machine
type state_type is (IDLE, LOAD, EXECUTE, STORE, DONE);
signal current_state, next_state : state_type;

-- State register
process(clk, reset)
begin
  if reset = '1' then
    current_state <= IDLE;
  elsif rising_edge(clk) then
    current_state <= next_state;
  end if;
end process;

-- Next state logic
process(current_state, start, load_done, exec_done, store_done)
begin
  case current_state is
    when IDLE =>
      if start = '1' then
        next_state <= LOAD;
      else
        next_state <= IDLE;
      end if;

    when LOAD =>
      if load_done = '1' then
        next_state <= EXECUTE;
      else
        next_state <= LOAD;
      end if;

    when EXECUTE =>
      if exec_done = '1' then
        next_state <= STORE;
      else
        next_state <= EXECUTE;
      end if;

    when STORE =>
      if store_done = '1' then
        next_state <= DONE;
      else
        next_state <= STORE;
      end if;

    when DONE =>
      next_state <= IDLE;
  end case;
end process;

-- Output logic
process(current_state)
begin
  -- Default outputs
  load_enable <= '0';
  exec_enable <= '0';
  store_enable <= '0';
  busy <= '1';

  case current_state is
    when IDLE =>
      busy <= '0';
    when LOAD =>
      load_enable <= '1';
    when EXECUTE =>
      exec_enable <= '1';
    when STORE =>
      store_enable <= '1';
    when DONE =>
      busy <= '0';
  end case;
end process;
```

Generic Programming

```
-- Generic entity
entity generic_counter is
  generic (
    WIDTH : positive := 8;
    MAX_VALUE : positive := 255;
    RESET_VALUE : natural := 0
  );
  port (
    clk   : in  std_logic;
    reset : in  std_logic;
    enable: in  std_logic;
    count : out std_logic_vector(WIDTH-1 downto 0);
    tc    : out std_logic  -- Terminal count
  );
end entity;

-- Generic instantiation
counter_8bit: entity work.generic_counter
  generic map (
    WIDTH => 8,
    MAX_VALUE => 255,
    RESET_VALUE => 0
  )
  port map (
    clk => system_clk,
    reset => system_reset,
    enable => count_enable,
    count => counter_value,
    tc => counter_tc
  );
```

Configurations

```
-- Configuration declaration
configuration counter_config of tb_counter is
  for behavioral
    for uut : counter
      use entity work.counter(behavioral);
    end for;
  end for;
end configuration;

-- Alternative configuration
configuration counter_rtl_config of tb_counter is
  for behavioral
    for uut : counter
      use entity work.counter(rtl);
    end for;
  end for;
end configuration;
```

# Common Patterns

Clock Domain Crossing

```
-- Double flip-flop synchronizer
entity synchronizer is
  port (
    clk_dest : in  std_logic;
    reset    : in  std_logic;
    async_in : in  std_logic;
    sync_out : out std_logic
  );
end entity;

architecture rtl of synchronizer is
  signal ff1, ff2 : std_logic;
  attribute ASYNC_REG : string;
  attribute ASYNC_REG of ff1 : signal is "TRUE";
  attribute ASYNC_REG of ff2 : signal is "TRUE";
begin
  process(clk_dest, reset)
  begin
    if reset = '1' then
      ff1 <= '0';
      ff2 <= '0';
    elsif rising_edge(clk_dest) then
      ff1 <= async_in;
      ff2 <= ff1;
    end if;
  end process;

  sync_out <= ff2;
end rtl;
```

Memory Inference

```
-- RAM inference
type ram_type is array (0 to 255) of std_logic_vector(7 downto 0);
signal ram : ram_type;

process(clk)
begin
  if rising_edge(clk) then
    if write_enable = '1' then
      ram(to_integer(unsigned(address))) <= data_in;
    end if;
    data_out <= ram(to_integer(unsigned(address)));
  end if;
end process;
```

Reset Strategies

```
-- Asynchronous reset
process(clk, async_reset)
begin
  if async_reset = '1' then
    output <= '0';
  elsif rising_edge(clk) then
    output <= input;
  end if;
end process;

-- Synchronous reset
process(clk)
begin
  if rising_edge(clk) then
    if sync_reset = '1' then
      output <= '0';
    else
      output <= input;
    end if;
  end if;
end process;
```

Top Cheatsheet
- Python Cheatsheet — Popular Quick Reference
- Vim Cheatsheet — Popular Quick Reference
- JavaScript Cheatsheet — Popular Quick Reference

Recent Cheatsheet
- Cheatsheet — Latest Quick Reference
- Socket io Cheatsheet — Latest Quick Reference
- GnuPG Cheatsheet — Latest Quick Reference

Developer Tools
- TableConvert — Convert tables between formats
- doRefer — Best referral codes & bonuses
- Fiona AI — Chrome AI assistant

cheatsheets.zip
Share quick reference and cheat sheet for developers.
About
Notes
Support

© 2025 cheatsheets.zip, All rights reserved.